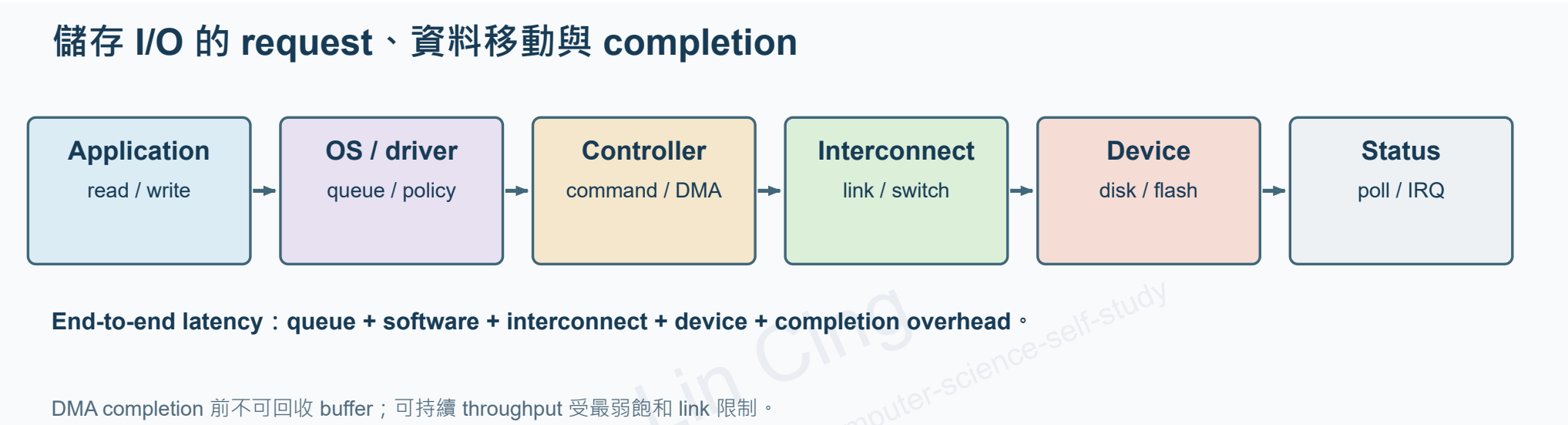
從一筆 request 看完整系統



讀檔會經過檔案系統、driver、controller 與儲存媒體；發出命令、資料抵達記憶體及應用程式可使用 buffer，是不同事件。分析效能時，先固定 request 大小、讀寫比例、queue depth、cache 狀態與完成的語意。揮發性 cache 回報寫入完成，不等於斷電後資料仍存在。

容量、頻寬與 IOPS 的分母

本章 GB、TB 分別採 $10^9, 10^{12}$ bytes；GiB、TiB 分別採 $2^{30}, 2^{40}$ bytes。Link 的 bit/s 須除以8才變成 byte/s，還需扣除協定額外流量。

若每個完成的request都是4 KiB，100,000 IOPS的payload頻寬是

$$100000 \times 4096 = 409600000\ \mathrm{B/s} = 0.4096\ \mathrm{GB/s}.$$

若改成64 KiB而IOPS不變，則是6.5536 GB/s，但裝置與link未必能承受。IOPS不是與request大小無關的常數。Response time從提交到完成；throughput是穩定狀態每秒完成數，不能取任意一筆latency的倒數代表並行系統吞吐量。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

6.1 依工作負載選設計目標

互動式查詢重視尾端延遲；連續備份重視持續頻寬與恢復能力；交易系統還要求commit後的持久性與失敗原子性。相同SSD數量可以因request pattern與完成條件不同而產生不同瓶頸。

設計的限制包括裝置、controller、interconnect、記憶體及CPU處理命令的能力。擴充裝置還會消耗電力、queue entries、interrupt資源及故障恢復時間。高峰值頻寬無法單獨保證低延遲或高可用度。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

6.2 Failure、reliability 與 availability

服務規格先定義可接受行為。實際服務偏離規格是failure（失效）；由失效狀態恢復符合規格是restoration（恢復）。元件fault可能被冗餘遮蔽，未必成為使用者看見的服務failure。

Reliability $R(t)$ 是從正常狀態起，至時間t仍連續正常的機率。Availability是觀察時刻服務正常的機率，或長期正常時間比例，允許中途失效後修復。修得很快可提高availability，卻不會把中途曾失效的歷史改成連續可靠。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

從更新週期推導可用度

假設系統反覆經歷正常時間U與修復時間D，各週期的(U_i,D_i)獨立同分布、非負，且0<E[U]+E[D]<∞。由大數法則，經歷k個完整週期後，正常比例為

$$A_k = \frac{\sum_{i=1}^{k} U_i}{\sum_{i=1}^{k} (U_i + D_i)} \longrightarrow \frac{E[U]}{E[U] + E[D]}.$$

因此 $A = MTTF/(MTTF + MTTR)$。這個長期比率不要求每次正常與修復時間都剛好等於平均值，也不給出每年最大停機次數。

MTTF=100,000小時、MTTR=4小時時，A=0.9999600016。8,760小時的長期平均停機量約為 $8760(1 - A) \times 60 = 21.02$ 分鐘。MTTR降到0.5小時後約2.63分鐘，減少18.39分鐘。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

平均失效時間不等於故障期限

只有另假設固定hazard λ 的指數壽命，才有

$$R(t) = e^{-\lambda t}, \qquad MTTF = 1/\lambda, \qquad P(T \le t) = 1 - e^{-t/MTTF}.$$

MTTF=100,000小時時，一年內失效機率在此模型下是 $1 - e^{-8760/100000} \approx 8.39\%$，不是保證可用100,000小時，也不是「一年最多一次」。元件老化、共同供電與環境損壞會破壞固定hazard或獨立性假設，不能只憑平均值外推實際風險。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

6.3 磁碟幾何與邏輯容量

磁碟由盤面、磁軌及sector（扇區）組成，同一磁頭半徑的各盤面磁軌稱cylinder。舊式固定幾何模型的payload容量為

$$C = \text{cylinders} \times \text{surfaces} \times \text{sectors/track} \times \text{bytes/sector}.$$

10,000 cylinders、4 surfaces、500 sectors/track、512 bytes/sector得到10,240,000,000 bytes，即10.24 GB或約9.5367 GiB。若這些已是可定址的logical sectors，不能再把raw媒體的gap/ECC比例任意扣一次。現代裝置使用LBA與控制器轉換，不把這個固定幾何公式當作實際物理配置。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

一筆 random read 的各段時間

忽略排隊時，存取時間為

$$T_s = T_{seek} + T_{rotation} + T_{transfer} + T_{controller}.$$

在12,000 RPM且每track500 sectors的模型，轉一圈 $60/12000 = 5$ ms；均勻隨機到達角度的平均旋轉等待2.5 ms；一sector佔 $5/500 = 0.01$ ms。平均seek4.5 ms、controller0.10 ms，得到

$$T_s = 4.5 + 2.5 + 0.01 + 0.10 = 7.11 \text{ ms}.$$

只有一筆outstanding且不重排時，長期完成率約 $1/0.00711 = 140.65$ IOPS。這不是「提高queue depth後吞吐量永遠不變」的定理，因為排程與locality可能改變平均service time。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Sequential request 如何攤提固定成本

同一磁軌上100個相鄰sectors合併成一筆request，明定不跨track、不另付setup及rotation。Transfer為 $100 \times 0.01 = 1$ ms，總時間

$$4.5 + 2.5 + 1 + 0.10 = 8.10 \text{ ms}.$$

Payload為51,200 bytes，平均6.321 MB/s。若拆成100筆互不相關random requests，模型時間是711 ms。差異來自只付一次定位成本，不是把媒體旋轉速度提高100倍。真實連續讀還要明定track crossing、預讀cache與controller pipeline。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

6.4 NAND 的 page 與 erase block

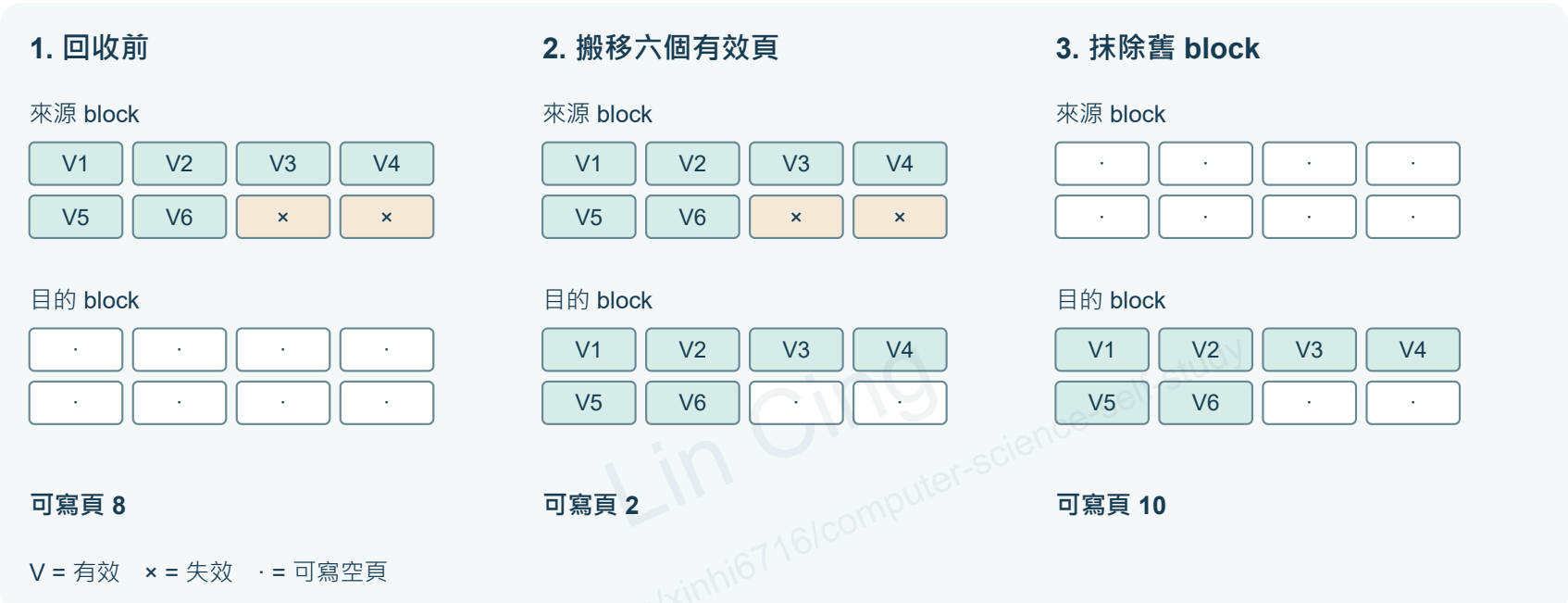
NOR適合較細粒度讀取與部分直接執行的firmware用途；NAND以較高密度支援大量儲存，通常按page讀寫、按較大的erase block抹除。兩者皆非揮發性，但介面、成本與更新粒度不同。已寫入NAND page不能像DRAM word一樣任意原地重寫；FTL（flash translation layer）將logical page映到新physical page，使舊page失效，再由garbage collection回收erase block。

設一個erase block有8個pages，其中6個有效、2個失效。回收時先把6個有效pages複製到其他空間，再erase舊block，產生8個可寫pages；扣掉保存舊資料所需的6個位置，本輪只淨增加2個可供host更新的位置。

若本輪2個新host pages加上6個搬移pages，NAND共program8 pages，則WA=8/2=4。只計這個循環，不把erase次數直接加成寫入bytes。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Garbage collection：逐頁追蹤



假設目的 block 原有 8 個可寫空頁，且不考慮其他 block 的變化。搬移前有 8 個空頁；複製六個有效頁後剩 2 個；抹除來源 block 後增加 8 個，合計 10 個。因此 GC 使系統的可寫空頁淨增加 2 個。

若接著 host 寫入 2 個同大小的新頁，這一輪共 program 6 個搬移頁與 2 個 host 頁，write amplification 為 $8/2 = 4$。這是特定批次的資料寫入比例；實際 WA 還受工作負載、有效頁比例及 metadata 影響。

Endurance 的粗估與限制

Write amplification定義為 $WA = \text{media寫入bytes} / \text{host寫入bytes}$。Host一天120 GB、media384 GB，WA=3.2。

若physical NAND400 GB、可用P/E預算3000 cycles，且均勻磨耗、WA固定、無其他保留預算，host總寫入量粗估

$$\frac{400 \times 3000}{3.2} = 375000 \text{ GB} = 375 \text{ TB}.$$

這不是產品保固。Wear leveling、壞block備援、溫度、retention與metadata都有額外影響。Overprovisioning增加可搬移空間，通常有助GC；若同時改變working set，觀察到的WA不能全歸因於保留比例。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

6.5 Link、bus 與端到端瓶頸

Shared bus的多個裝置競爭同一傳輸資源；point-to-point link與switch可平行不同路徑，但仍可能共用上游link、controller與memory channel。裝置總峰值不能直接相加成應用吞吐量。

例如6個SSD各80,000 IOPS、HBA120,000、memory path300,000、CPU/driver150,000 IOPS，穩態上界為

$$\min(6 \times 80000, 120000, 300000, 150000) = 120000.$$

再加兩個SSD仍受HBA限制。若HBA升到240,000，下一個上界變成CPU的150,000。這些是固定同一request大小與讀寫mix的容量上界，排隊、lock與error recovery可讓實測更低。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

分離交易：等待時釋放 bus

分離交易：等待期間讓出共享 bus



A: ID 1　request [0,1) → response [6,7)　latency = 7 slots

B: ID 2　request [1,2) → response [4,5)　latency = 4 slots

同一時槽至多一筆 bus 傳輸；回應以 transaction ID 配對。

假設每個 request／response 占一個時槽；目標裝置可同時處理兩筆請求，且 bus 每次只能傳一筆。A 先發出、B 後發出，但 B 先完成；transaction ID 讓回應即使重排也能送回正確的 requester。

圖中 7 個時槽有 4 個用於傳輸；A 的端到端延遲是 7，B 是 4。釋放 bus 讓等待與其他工作重疊，**不保證單筆延遲降低**；競爭、仲裁與裝置是否支援多筆未完成交易都會改變結果。

6.6 命令、通知與資料搬移彼此獨立

MMIO（memory-mapped I/O）把裝置register映到位址空間，由具適當權限的load/store存取。特定ISA也有獨立I/O指令。裝置register的cacheability與ordering由平台規格決定，不能當作普通可快取RAM。

Polling與interrupt回答「如何得知狀態」；programmed I/O與DMA回答「誰搬資料」。DMA可搭配poll completion，也可搭配interrupt completion。使用interrupt不表示每個data word都由CPU搬移。

Doorbell store只是通知裝置有新descriptor。若descriptor資料尚未對裝置可見，裝置可能讀到舊位址或舊長度；正確順序須依OS DMA與I/O API建立，不是只加一條任意store即可。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Polling 與 interrupt 的實算

3 GHz CPU，每秒20,000筆I/O。若每筆恰好一次poll、每poll100 cycles，成本為2,000,000 cycles/s，即0.0667% CPU。每筆interrupt handler1500 cycles時，成本30,000,000 cycles/s，即1% CPU。

這個一次poll的模型中，增加runnable threads數不會改變上述算式。若每次提交後忙等50 µs，且不與其他requests共用等待，20,000筆已需要整個core的1秒等待時間；不能沿用「只一次poll」的0.0667%結論。

Hybrid可先短暫poll再sleep，completion queue也可批次處理多筆。比較時需計setup、忙等、context switch、completion批次大小及延遲分布，不能先假設interrupt永遠較快。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

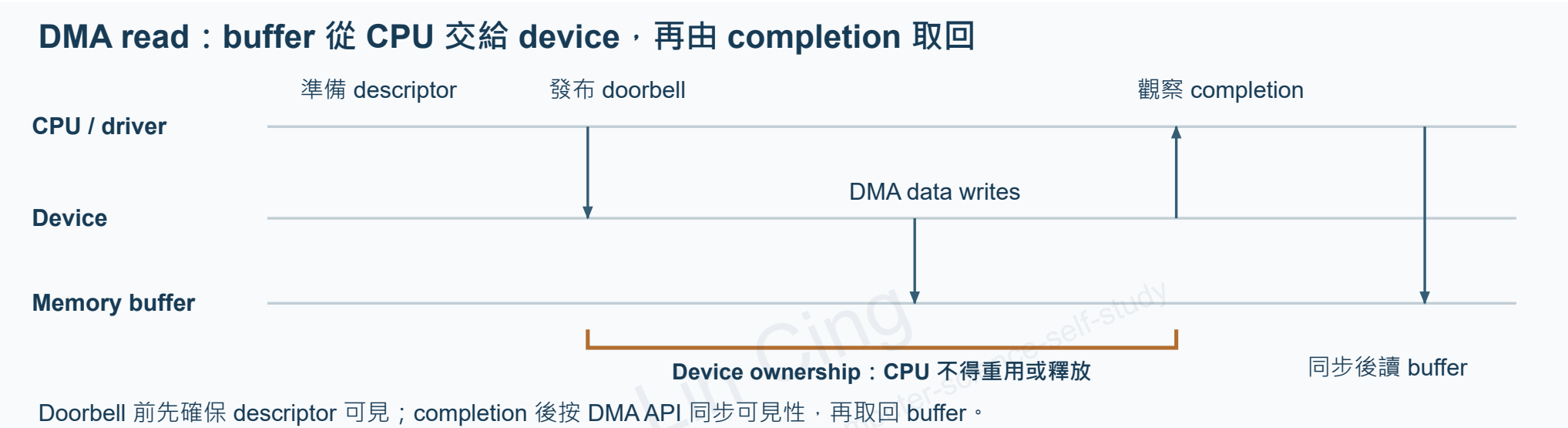
Interrupt priority 與完成處理

Device設pending狀態，controller/CPU依enable與priority決定是否接受interrupt。進入handler後，driver讀status、確認來源、處理completion/error、按裝置規則清除或acknowledge來源，再喚醒等待工作。若一個shared interrupt有多個可能來源，不能無條件假設每次都是自己的device。

假設低priority handler關閉全部interrupt共8 μs，高priority事件在關閉後1 μs到達，則單是masking便使它至少等7 μs；「它priority較高」不能消除這段等待。允許nested interrupts可減少某些等待，但需保存獨立context、保護共享driver狀態並界定最大巢狀深度。Coalescing合併多個completion可降低每秒handler次數，也可能增加第一個completion的通知等待。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

DMA 的 buffer ownership 狀態



裝置寫入記憶體前，CPU 配置 buffer 並發布 descriptor；裝置持有期間，CPU 不可釋放、重用或讀取未完成內容。完成後 CPU 依 DMA API 同步可見性，再取回 buffer。

非一致性平台須在正確時點 clean／invalidate：DMA 寫完後才寫回舊的 dirty cache line，可能覆蓋新資料。Timeout 不代表裝置已停止；解除映射前須確認不會再有 DMA 寫入。

DMA 節省的是 core 搬移成本

64 MiB在20 ms完成，payload頻寬為

$$64 \times 2^{20} / 0.020 = 3.3554432 \ \mathrm{GB/s}.$$

CPU setup500 cycles在3 GHz需0.1667 µs。相較之下，programmed I/O每16 bytes需16搬移cycles加10 transaction cycles，共

$$(64 \times 2^{20} / 16) \times 26 = 109051904 \ \mathrm{cycles},$$

約36.35 ms CPU時間，尚不含等待。DMA仍消耗interconnect、memory bandwidth、IOMMU查詢與completion處理，不能把0.1667 µs誤當完整端到端時間。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

6.7 Little 定律連結 concurrency 與 latency

穩定、無持續累積工作的系統，平均系統內requests數 L、完成率 λ、平均response time W 滿足

$$L = \lambda W.$$

理由是觀察區間內「系統內人數對時間的面積」等於各request逗留時間之和；除以觀察長度，再讓起終邊界殘留的影響消失，即得此式。L含正在服務與排隊者；若只算queue，應使用Lq與Wq。

100,000 IOPS且平均response80 µs時，L=8。這不是保證把queue depth設8便達到該吞吐量，因為調整concurrency本身可能改變response time；它是同一穩態量測的必須一致關係。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

M/M/1 模型為何接近滿載就變慢

本頁另明定Poisson到達率λ、獨立指數service rate μ、單一server、FCFS、無限等待區，且ρ=λ/μ<1。這是分析模型，不宣稱磁碟實際service distribution一定指數。

平衡方程 $p_n\lambda = p_{n+1}\mu$ 給 $p_n = p_0\rho^n$。用機率和為1得 $p_0 = 1-\rho$。微分幾何級數可得

$$L = (1-\rho)\sum_{n\geq 0} n\rho^n = \frac{\rho}{1-\rho}, \quad W = L/\lambda = \frac{1}{\mu-\lambda}.$$

因此平均service time $1/\mu$ 不變時，queueing也能讓response暴增。ρ>=1不存在上述有限穩態平均，不能把負值或無窮級數公式硬套成正常latency。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

同一裝置兩個負載的排隊成本

取μ=200 IOPS，平均service5 ms。

λ	ρ	W=1/(μ−λ)	Wq=W−5 ms	L=λW
100 IOPS	0.5	10 ms	5 ms	1
160 IOPS	0.8	25 ms	20 ms	4
190 IOPS	0.95	100 ms	95 ms	19

從160增至190 IOPS只多18.75%完成量，平均response卻由25變100 ms。這是此模型的計算結果，不是所有實機固定倍數。只報最高IOPS會隱藏滿載附近的延遲代價；應同時固定平均/尾端latency限制。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

6.8 固定成本與持久性納入設計

總latency在無重疊串行路徑可相加；不同資源的持續吞吐上界取最小值。CPU與裝置可重疊時，不能把全部CPU時間和全部I/O時間無條件相加，也不能假設完全重疊。

Write-back cache可縮短回報時間，但若應用要求斷電後保留，就須把持久媒體、保護的cache或明確flush completion納入終點。兩個系統若一個回報「已入RAM」、另一個回報「已持久」，不能直接以latency比較相同服務。

錯誤重試需辨識命令是否已部分完成。重送write可能安全，重送「增加counter」卻可能執行兩次；controller/driver與上層交易協定共同決定恢復語意。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

6.9 RAID 的容量與容錯模型

本章以n顆等容量C的disks，不含spare、metadata及保留空間。Failure指可識別的整顆disk erasure，假設survivors資料正確、controller與供電正常。

配置	Usable容量	可恢復的整顆失效
RAID0 striping	nC	無冗餘
RAID1配對/RAID10	nC/2，n為偶數	每組mirror最多失一顆
RAID4/5	(n−1)C	任一顆
RAID6 P+Q	(n−2)C	任意兩顆，編碼及前提成立時

Lin Cing https://github.com/xinhi6716/computer-science-self-study

RAID 0、5、10、6 的實體磁碟 layout

RAID 0	RAID 5	RAID 10	RAID 6
disk0 D0/D4	disk0 D0/D3	disk0 D0/D2	disk0 D0/D2
disk1 D1/D5	disk1 D1/D4	disk1 D0/D2	disk1 D1/P1
disk2 D2/D6	disk2 D2/P1	disk2 D1/D3	disk2 P0/Q1
disk3 D3/D7	disk3 P0/D5	disk3 D1/D3	disk3 Q0/D3

每個 RAID 框內：各行是不同 disk，/ 分隔兩條 stripes；RAID10 跨 disk 鏡射，P/Q 為獨立檢查符號。

RAID 容忍指定 device failures，不取代 backup；重建流量與故障暴露窗口另計。

RAID3以細粒度bit/byte條帶配合同步存取，RAID4以block條帶讓小讀取分散，但集中parity造成寫入瓶頸。RAID5輪替parity位置，只消除固定單顆parity disk的瓶頸，並未消除parity更新成本。

XOR parity 的恢復證明與數值

以bitwise XOR定義 $P = D_0 \oplus D_1 \oplus D_2$。因為 $x \oplus x = 0$、$x \oplus 0 = x$ 且可交換結合，遺失 D_1 時

$$P \oplus D_0 \oplus D_2 = D_1.$$

取4-bit資料D0=1010、D1=1100、D2=0110，P=0000。遺失D1時，0000 XOR 1010 XOR 0110=1100，還原原值。

只有一個parity時，兩個未知data blocks只給一條XOR方程，因此通常不唯一。若disk回傳未被發現的錯誤值，甚至無法只由parity判定哪顆壞；「可恢復一個已知遺失」與「定位任意silent corruption」是不同保證。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Small write 與 full-stripe write

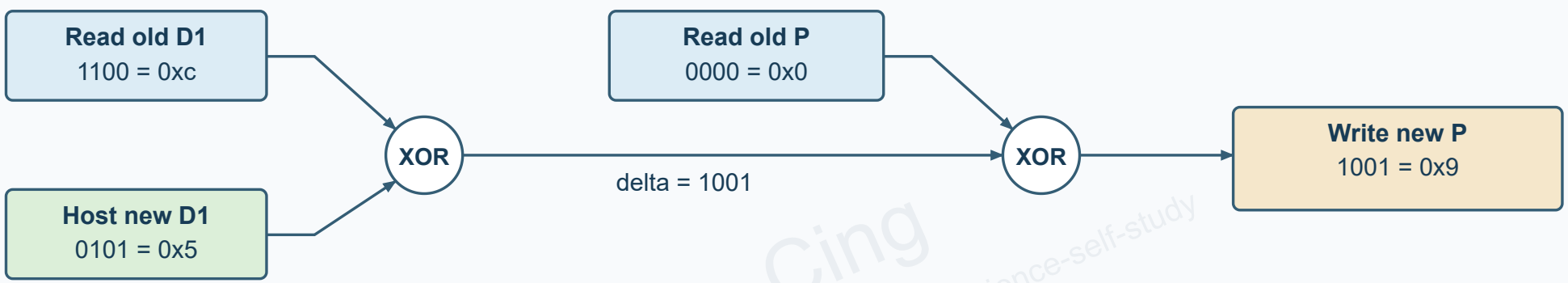
更新一個data block時，利用

$$P' = P \oplus D_{old} \oplus D_{new}$$

可讀old data、old parity，再寫new data、new parity，共2讀2寫。若上述D1由1100改成0101，P'=0000 XOR 1100 XOR 0101=1001。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Small write 與 full-stripe write （續）



若一次覆寫整條stripe的m個data blocks，全部new data都由host提供，可以直接算new parity，不需讀old blocks，只寫m個data加1個parity。若只更新k個data，reconstruct-write可讀其餘m−k個舊data再寫k個new data與parity；需比較 $2k+2$ 與 $(m-k)+(k+1)=m+1$ 次block I/O，並考慮cache及並行瓶頸。

RAID5 讀寫 mix 的容量上界

假設8顆disk，每顆持續100個block I/O/s；小讀只占1次disk I/O，小寫採2讀2寫，所有parity/data流量均衡，沒有cache合併或rebuild。令host中讀比例r，則每host request平均需要

$$r + 4(1 - r) = 4 - 3r$$

個disk I/O，因此host完成率 $X \leq 800/(4 - 3r)$。r=0.75時，平均1.75個，X<=457.14 IOPS。

這是aggregate工作量上界，並未保證每次small write四個操作可完全平行。若某顆disk、stripe lock或controller已先飽和，實測更低。RAID4另有集中parity disk：每small write該disk要一讀一寫，寫入率先受其能力除以2限制。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

兩份相同 parity 為何不是 RAID6

若Q只是P的另一份copy，遺失兩個data時，兩條方程完全相同，仍無法唯一恢復。RAID6需要獨立的檢查資訊。

一種P+Q模型是在有限域GF(2^w)逐symbol計算

$$P = \sum_i D_i, \qquad Q = \sum_i a_i D_i,$$

其中加法是XOR，非零係數a_i互異。遺失Di、Dj時，扣掉已知survivors得到 $X = D_i + D_j$、$Y = a_i D_i + a_j D_j$。因 $a_i + a_j \neq 0$ 有乘法逆元素，

$$D_j = (Y + a_i X)/(a_i + a_j), \qquad D_i = X + D_j.$$

這說明兩條獨立方程如何解出兩個erasure；不是把一般整數乘除直接套進bitwise XOR。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

P+Q 的一個可重算例子

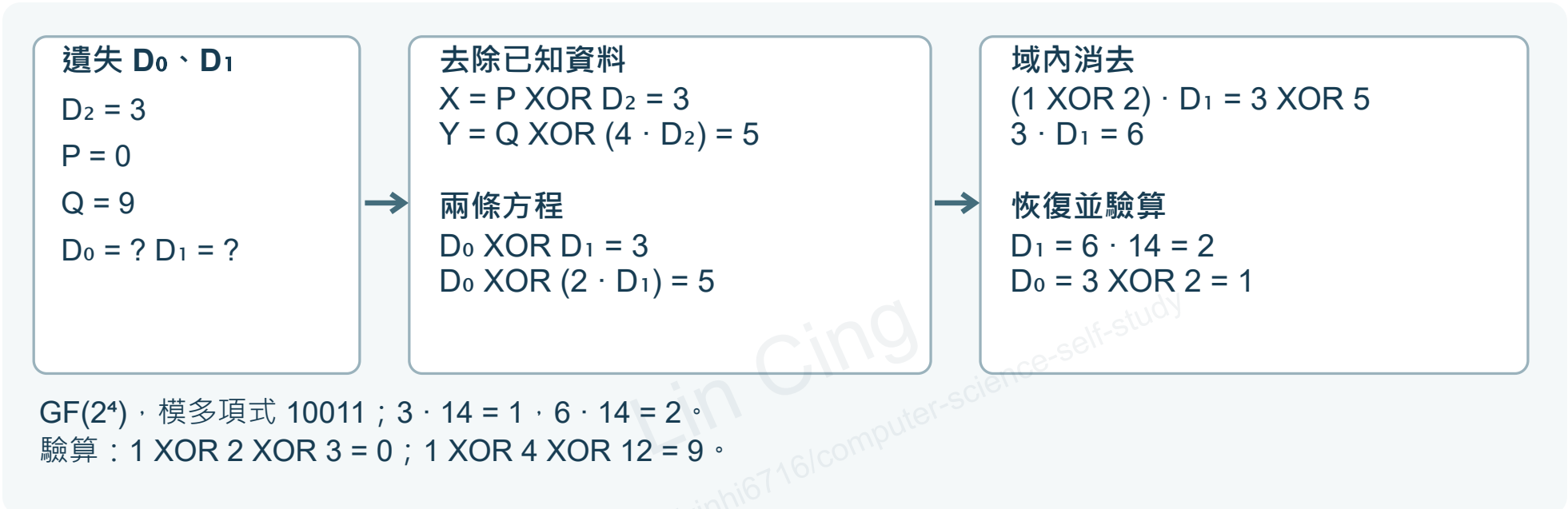
採GF(2^4)，模多項式 $x^4 + x + 1$（binary10011），symbol以4-bit多項式表示。乘2是左移；若原最高bit為1，截成4 bits後XOR0011。係數a0=1、a1=2、a2=4。

取D0=1、D1=2、D2=3。P=1 XOR 2 XOR 3=0；Q=(1・1) XOR (2・2) XOR (4・3)=1 XOR 4 XOR 12=9。

若D0、D1同時遺失，X=P XOR D2=3；Y=Q XOR(4・D2)=9 XOR 12=5。故3・D1=Y XOR X=6。域內3的逆為14，因對多項式模10011運算可得3・14=1；6・14=2，得D1=2、D0=3 XOR 2=1。實際RAID採用的symbol大小、係數與layout可不同，本例只證明明定編碼的恢復步驟。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

P+Q：兩個遺失值如何解出



GF(2^4)，模多項式 10011；$3 \cdot 14 = 1 \cdot 6 \cdot 14 = 2$。
驗算：1 XOR 2 XOR 3 = 0；1 XOR 4 XOR 12 = 9。

已知 D_2、P、Q，先從兩個 parity 扣除 D_2 的貢獻，再將兩條方程 XOR，消去 D_0。所有加法是 XOR，乘法及逆元素都在指定的有限域中；一般整數除法不能替代域運算。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Parity write hole 與一致性

Small write的data與parity是兩個實體寫入，若電源在兩者之間中斷，survivors可能各自讀取正常卻不屬於同一版本。例如old D1=1100、P=0000，先寫new D1=0101後中斷，parity仍0000，已不等於全部data的XOR。

這稱write hole。單靠parity公式無法保證更新原子性。受保護write cache、journal、完整stripe交易與恢復協定可保存重作資訊；各自需定義寫入順序、持久點及開機恢復規則。RAID也不能恢復合法指令造成的誤刪，因此不能取代獨立備份。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Rebuild 的流量分母

12顆10 TB的RAID5，明定每顆有效9 TB，controller只重建有效區域。重建每個遺失block要讀11個surviving blocks，因此讀99 TB、寫replacement9 TB。

若所有讀與寫共用aggregate1 GB/s路徑，時間下界

$$T \geq (99+9)10^{12}/10^{9} = 108000\ \mathrm{s} = 30\ \mathrm{h}.$$

若「1 GB/s」改指replacement輸出速率，則9 TB/1 GB/s=2.5小時，但surviving讀取路徑需同時支援11 GB/s。兩個分母不能混用。若全盤重建而不辨識有效區域，讀110 TB加寫10 TB，在aggregate路徑需至少33.33小時。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Rebuild：流量與頻寬要配對



每寫回一個遺失 block，就要讀取其餘 11 個 block，因此重建 9 TB 需要讀 99 TB、寫 9 TB。若同一條路徑承擔讀寫，分子是兩者的總流量；若 1 GB/s 專指 replacement 的寫入，分子只有 9 TB，但同時需要足夠的讀取頻寬。

以上都是理想化的下界，忽略競爭、故障處理與額外開銷；不能用單一鏈路數值代替整條路徑的容量。

重建期間的第二顆失效機率

明定n=8的RAID5已失去一顆，修復時間固定r=10小時，其餘7顆獨立、每顆固定hazardλ=10^−5/小時。剩餘disk全部活過r的機率為

$$(e^{-\lambda r})^7 = e^{-7\lambda r},$$

所以這段時間至少再失去一顆的機率

$$1 - e^{-7\times 10^{-5}\times 10} \approx 0.000699755,$$

約0.06998%。這是已進入降級狀態後的條件機率，不是array每年資料遺失率。讀取不可校正錯誤、共同故障、修復時間分布與再次故障需另建模，不能把此值稱完整RAID可靠度。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

6.10 Sun Fire 歷史案例與功耗邊界

原書Sun Fire x4150是2008年前後的1U server案例，列出的最大配置有兩個four-core Xeon sockets、64 GB DRAM、8顆15,000 RPM SAS disks及RAID controller。這些是原書的歷史配置，不代表當代產品。19-inch rack的1U高度為1.75 inches，42U為73.5 inches，還需留供電與散熱能力。

以明定的教學配置而非產品實測，設server idle200 W、滿載400 W、完成2000 transactions/s。滿載每transaction能耗400/2000=0.2 J。若半載仍耗300 W、只完成1000/s，則0.3 J/transaction。把兩部半載整合到一部滿載，只有在容量、尾延遲、容錯與遷移成本可接受時才可能減少總能耗。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

同一歷史配置，不同 workload 的瓶頸

以下另定義教學workload，並非原書效能量測。8個cores各持續10^9 instructions/s，每筆I/O需250,000 user加150,000 OS instructions；每筆64 KiB。CPU上界為8×10^9/400000=20000 IOPS。

假設每disk的random-read平均service4 ms，8顆合計2000 IOPS；sequential每disk持續112 MB/s，合計 $8 \times 112 \times 10^6 / 65536 = 13671.875$ IOPS。單一RAID card的payload link若2 GB/s，則 $2 \times 10^9 / 65536 = 30517.578$ IOPS。

在忽略其他限制、無disk conflict的模型，random上界min(20000,2000,30517.578)=2000，sequential上界13671.875，兩者皆disk先限制。若把裝置換成更快SSD超過20000 IOPS，CPU便可能先限制；歷史配置的圖與新模型的workload不能混成同一份產品benchmark。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

6.11 Network 的傳輸與等待

10 Gbit/s link、payload efficiency80%，有效頻寬8 Gbit/s=1 GB/s。傳64 MiB需

$$64 \times 2^{20}/10^{9} = 67.108864\ \text{ms}.$$

若每4 MiB為一batch，16 batches之間各需0.20 ms gap，且連線已建立，則15 gaps共3 ms，總70.108864 ms。100條streams可以重疊某些等待，卻不能把共用link的總payload能力變成100 GB/s。

Propagation（訊號傳播）、serialization（把bits送入link）、queueing與protocol setup是不同項。Bandwidth-delay product為頻寬乘RTT，表示維持某速率所需的在途資料量，而不是每筆request自動增加的固定資料量。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

6.12 用反例檢查常見推論

「DMA不耗CPU」漏掉setup與completion，且資料仍占memory bandwidth。「RAID6有兩個parity blocks就安全」漏掉獨立編碼與write hole。「MTTF高就一年不會壞」把平均值誤當期限。

「IOPS相同所以效能相同」可由4 KiB與64 KiB反例推翻。「提高queue depth一定降低response time」可由M/M/1表推翻。「RAID就是backup」則無法處理誤刪及攻擊者同步改寫所有copies。

每個反例指出失效的前提，不把特定workload的結論外推成所有裝置或所有版本。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

6.13 從觀測值回推限制

若吞吐不再上升而response增加，先檢查各資源利用率、queue length與per-I/O成本。Little定律可檢查三個量是否來自同一穩態，但不能單獨定位哪一段飽和。

若寫入回報很快卻在斷電後遺失，應檢查完成語意與持久路徑；若RAID重建超時，應分開計surviving讀、replacement寫與共用link。故障與效能分析都從清楚的端到端服務規格開始。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

6.14 技術演進與不變的分析方法

磁碟的機械定位、NAND的out-of-place更新與互連的共享資源不同，但都需分開容量、service rate、queueing與failure model。由bus轉向switch、由HDD轉向SSD，只改變主要成本，不消除buffer ownership、completion及持久性問題。

原書的歷史數據適合觀察設計取捨；比較新系統時必須重新取得相同workload與完成條件的數據，不把2008年的裝置峰值當作現代通用常數。

參考資料與署名

David A. Patterson、John L. Hennessy，《Computer Organization and Design: The Hardware/Software Interface》，Revised Fourth Edition，第 6 章。Lin Cing，《計算機組織》手寫筆記；計算機組織與結構（CSO）課程投影片。

Lin Cing 原創教學整理，原創內容以 CC BY 4.0 授權。

https://github.com/xinhi6716/computer-science-self-study